

Introduction

Large Language Models (LLMs) such as ChatGPT have demonstrated remarkable capabilities in natural language understanding and generation. However, their effectiveness is constrained by the context window limitations and the absence of domain-specific or up-to-date information during inference. Retrieval-Augmented Generation (RAG) addresses these limitations by integrating external knowledge retrieval mechanisms with generative models.

RAG operates on the principle of augmenting the input prompt with relevant contextual information retrieved from a curated knowledge base. This approach enables the model to generate more accurate, contextually grounded, and factually consistent responses. The retrieval process involves querying a document corpus based on semantic similarity to the user's input, followed by the incorporation of the most relevant passages into the prompt provided to the LLM.

This project presents an implementation of a RAG system featuring a JavaFX-based graphical user interface for knowledge base management. The system provides functionality for resource ingestion, organization, and retrieval, enabling users to maintain a structured repository of documents and articles. Upon receiving a query, the system employs semantic search techniques to identify relevant documents from the knowledge base, which are subsequently utilized to augment the LLM prompt.

The implementation incorporates advanced techniques including response streaming, multisource retrieval, context chaining, and re-ranking mechanisms. The subsequent sections detail the architectural design, implementation methodology, and evaluation of the proposed RAG system.

Problem Definition & Objectives

Problem Definition

Traditional Large Language Models (LLMs) face several critical limitations when deployed in practical applications. First, they are constrained by fixed training data cutoffs, making them unable to access information beyond their training period. Second, their context window limitations restrict the amount of information that can be processed in a single query. Third, they lack domain-specific knowledge that may be proprietary or specialized to particular organizations or fields of study.

These limitations manifest as several concrete problems:

- **Hallucination:** LLMs may generate plausible-sounding but factually incorrect information when queried about topics outside their training data
- **Lack of Source Attribution:** Users cannot verify the factual basis of generated responses
- **Inability to Access Private Data:** Organizations cannot leverage LLMs with their internal documentation and knowledge bases
- **Static Knowledge:** Models cannot be updated with new information without expensive retraining

The problem is particularly acute in educational settings where students need to query course materials, lecture notes, and academic resources. A typical scenario involves a student wanting to ask questions about their course content, but generic LLMs cannot access these specific materials, leading to irrelevant or incorrect responses.

Purpose & Objectives

The primary purpose of this project is to develop a Retrieval-Augmented Generation (RAG) system that addresses the aforementioned limitations by combining the generative capabilities of LLMs with dynamic knowledge retrieval mechanisms.

Primary Objectives:

1. **Knowledge Base Management:** Enable users to create, organize, and manage multiple knowledge bases (sessions) containing documents relevant to specific topics or courses
2. **Semantic Document Retrieval:** Implement vector-based semantic search to retrieve contextually relevant document segments based on user queries
3. **Context-Aware Query Processing:** Maintain session history to handle follow-up questions and contextual references
4. **Response Re-ranking:** Improve retrieval quality through hybrid scoring that combines semantic similarity with lexical matching
5. **Source Attribution:** Provide transparent citations of source documents used in generating responses
6. **Multi-Model Support:** Allow users to select from different OpenAI models (GPT-4o, GPT-4.1, GPT-5 series) based on their requirements

Secondary Objectives:

1. Provide an intuitive JavaFX-based graphical interface for non-technical users
2. Implement persistent storage for session history and embeddings to reduce computational overhead
3. Support multiple document formats (PDF, TXT, DOC, DOCX, PPT, PPTX)
4. Enable session-based organization for managing different knowledge domains
5. Implement efficient caching mechanisms to minimize API costs and latency

Targeted Users

The system is designed for the following user groups: **Primary Users:**

- **Students:** Individuals who need to query course materials, lecture notes, and academic resources for studying and assignment completion
- **Researchers:** Academics who want to ask questions about research papers and technical documentation
- **Knowledge Workers:** Professionals who need to reference internal documentation, policies, and procedures

Secondary Users:

- **Educators:** Teachers who want to create knowledge bases from course materials for student access
- **Content Curators:** Individuals responsible for organizing and maintaining organizational knowledge repositories

User Characteristics:

- May have limited technical expertise in AI/ML
- Require intuitive interfaces for document management
- Need reliable, cited responses rather than speculative answers
- Work with domain-specific or proprietary information not available to public LLMs

System Scope

The implemented RAG system encompasses the following functional scope:

Included Features:

1. Session Management:

- Create multiple independent knowledge bases (sessions)
- Edit session names and associated AI models
- Delete sessions with confirmation safeguards
- Persistent storage of session metadata

2. Knowledge Base Management:

- Import documents in multiple formats (PDF, TXT, DOC, DOCX, PPT, PPTX)
- View document content before import
- Replace existing documents with updated versions
- Remove documents from knowledge bases
- Automatic text extraction and preprocessing

3. Intelligent Query Processing:

- Semantic search using OpenAI embeddings (text-embedding-3-small)
- Session-aware query contextualization
- Hybrid re-ranking combining semantic and lexical signals
- Retrieval of top-5 most relevant document segments
- Source attribution for transparency

4. Sessional Interface:

- Multi-turn session support with context retention
- Session history persistence across sessions
- Clear session functionality
- Real-time response generation with loading indicators
- Display of source documents for each response

5. Performance Optimization:

- Embedding caching to avoid redundant API calls
- Incremental indexing (only process new/modified files)
- Session-isolated embedding stores for memory efficiency

The following features are out of scope for the current implementation:

- Web-based deployment (desktop application only)
- Multi-user collaboration features
- Document editing capabilities within the application
- Support for non-textual documents (images, audio, video)
- Custom embedding model training
- Integration with databases or enterprise systems

Requirement Specifications

This section delineates the comprehensive functional and non-functional requirements that guided the development of the RAG system. The requirements were derived from user needs analysis and technical feasibility considerations.

Functional Requirements

FR1: Session Management

FR1.1 - Session Creation

- The system shall allow users to create new sessions with user-defined names
- Each session shall be assigned a unique identifier upon creation
- New sessions shall default to the GPT-4o-mini model
- Session creation shall initialize an empty knowledge base and session history

FR1.2 - Session Selection

- The system shall display all existing sessions in a sidebar interface
- Users shall be able to select a session by clicking on it
- Upon selection, the system shall load the associated knowledge base and session history
- The main interface shall display the selected session's name and model

FR1.3 - Session Editing

- Users shall be able to edit session names through a dialog interface
- Users shall be able to change the AI model associated with a session
- The system shall provide selection from eight OpenAI models: gpt-4o-mini, gpt-4o, gpt-4.1, gpt-4.1-mini, gpt-4.1-nano, gpt-5, gpt-5-mini, gpt-5-nano
- Model changes shall trigger reinitialization of the RAG service
- Changes shall be persisted to the database immediately

FR1.4 - Session Deletion

- The system shall provide a delete function for sessions
- Deletion shall require explicit user confirmation via dialog
- Deleting a session shall cascade delete all associated messages and embeddings
- The system shall prevent deletion of the currently selected session without warning

FR2: Knowledge Base Management

FR2.1 - Document Import

- The system shall support import of documents in the following formats: PDF, TXT, DOC, DOCX, PPT, PPTX
- Users shall select documents through a file chooser dialog
- The system shall extract text content from all supported document types
- Extracted text shall be split into chunks for embedding generation

FR2.2 - Document Preview

- Users shall be able to view the full text content of any document before import
- The preview shall be displayed in a modal dialog with scrollable text area
- The preview shall accurately represent the text that will be indexed

FR2.3 - Document Replacement

- If a document with the same filename already exists, the system shall prompt the user
- Users shall have the option to replace the existing document or cancel the import
- Replacement shall delete old embeddings and generate new ones

FR2.4 - Document Deletion

- Users shall be able to remove documents from a knowledge base
- Deletion shall require explicit confirmation
- The system shall remove both the document file and associated embeddings
- Deletion shall update the embedding store index

FR2.5 - Document Listing

- The system shall display all documents in the selected session's knowledge base
- Each document entry shall show: filename, file format icon, and action buttons
- The interface shall support scrolling for knowledge bases with many documents

FR3: Query Processing & Response Generation

FR3.1 - Query Input

- Users shall enter queries through a text input field
- The system shall support multi-line queries
- The send button shall be disabled when the input field is empty
- Queries shall be limited to reasonable length to prevent API errors

FR3.2 - Contextual Query Understanding

- The system shall incorporate the last four messages (two exchanges) from session history into the retrieval query
- This contextualized query shall be used for semantic search only
- The original user message shall be preserved for display and re-ranking

FR3.3 - Semantic Retrieval

- The system shall embed the contextualized query using OpenAI's text-embedding-3-small model
- The system shall retrieve the top 15 most semantically similar document segments
- Retrieval shall be performed against the session-specific embedding store

FR3.4 - Hybrid Re-ranking

- Retrieved segments shall be re-ranked using a hybrid scoring algorithm
- The scoring shall combine: semantic similarity (60%), term frequency (30%), position bonus (5%), exact phrase matching (5%)
- The top 5 segments after re-ranking shall be selected for context augmentation

FR3.5 - Response Generation

- The system shall construct a prompt containing the retrieved context segments
- The prompt shall include the full session history for context awareness
- The system shall send the request to the OpenAI Chat API using the selected model
- Responses shall be streamed and displayed in real-time

FR3.6 - Source Attribution

- Each response shall include citations of source documents used
- Citations shall display the document filename
- Users shall be able to identify which documents contributed to the response

FR4: Session Management

FR4.1 - Session History Display

- The system shall display session history in a scrollable chat interface

- User messages shall be visually distinguished from AI responses
- Each message shall include appropriate styling and alignment

FR4.2 - Session Persistence

- All messages (user and AI) shall be saved to the database with timestamps
- Session history shall be automatically loaded when a session is selected
- The system shall restore the complete session state from the database

FR4.3 - Session Clearing

- Users shall be able to clear all messages in the current session
- Clearing shall require explicit confirmation via dialog
- The operation shall delete messages from the database and remove them from the UI
- The RAG service's internal session history shall be reset
- The knowledge base shall remain intact

FR4.4 - Context Chaining

- The system shall maintain a clean session history without embedded RAG context
- RAG context shall be injected only into the current query for generation
- Subsequent queries shall reference the clean session history

FR5: User Interface Controls

FR5.1 - Control State Management

- All input controls shall be disabled during AI response generation
- This includes: message input field, send button, manage knowledge base button, clear session button, and session sidebar
- Controls shall be re-enabled after response completion or error

FR5.2 - Conditional Visibility

- The manage knowledge base button shall be hidden when no session is selected
- The clear session button shall be hidden when no session is selected
- Hidden buttons shall not occupy layout space

FR5.3 - Progress Indicators

- The system shall display a loading indicator during document processing
- Progress dialogs shall show operation status for long-running tasks
- Users shall receive feedback for all system operations

Non-Functional Requirements

NFR1: Performance

NFR1.1 - Embedding Cache Efficiency

- The system shall cache embeddings to avoid redundant API calls
- Only new or modified documents shall trigger embedding generation
- Cache lookup shall be performed before any API call

NFR1.2 - Memory Optimization

- Each session shall maintain an isolated embedding store
- Embedding stores shall be loaded on-demand when sessions are selected

NFR2: Reliability

NFR2.1 - Data Persistence

- All session data, messages, and metadata shall be persisted to SQLite database
- Database operations shall use transactions to ensure consistency
- The system shall handle database connection failures gracefully

NFR2.2 - Error Handling

- All API errors shall be caught and reported to the user with descriptive messages
- File I/O errors shall not crash the application
- Invalid document formats shall be rejected with appropriate error messages

NFR2.3 - Data Integrity

- Cascading deletes shall ensure no orphaned records in the database
- Foreign key constraints shall be enforced
- The system shall validate user input before database operations

NFR3: Usability

NFR3.1 - User Interface Design

- The interface shall be clean and intuitive
- All interactive elements shall provide visual feedback on hover and click
- Error messages shall be clear and actionable

NFR3.2 - Confirmation Dialogs

- All destructive operations (delete, replace, clear) shall require explicit confirmation
- Confirmation dialogs shall clearly state the consequences of the action
- Users shall have the option to cancel destructive operations

NFR3.3 - Visual Hierarchy

- The interface shall use consistent spacing, colors, and typography
- Primary actions shall be visually prominent
- The chat interface shall clearly distinguish user messages from AI responses

NFR4: Maintainability

NFR4.1 - Code Organization

- The system shall follow MVC architectural pattern
- Business logic shall be separated from UI code
- Commonly used services shall be implemented as singletons for global access whenever appropriate

NFR4.2 - Configuration Management

- System constants shall be centralized in a Constants class
- API keys shall be stored securely in environment variables and not hardcoded
- Model configurations shall be easily modifiable

NFR4.3 - Logging

- The system shall use Log4j2 for structured logging
- Error logs shall capture stack traces and context information

- Logs shall be written to console during development for real-time monitoring, as well as to files for debugging

NFR5: Security & Privacy

NFR5.1 - API Key Management

- OpenAI API keys shall be stored in environment variables (.env file)
- The system shall prompt for API key if not configured
- API keys shall not be logged or displayed in plain text
- API key validity shall be verified on application startup through a test request to OpenAI
- Invalid API keys shall trigger an error dialog and disable chat functionality

NFR5.2 - Data Privacy

- All user data shall be stored locally on the user's machine
- No data shall be transmitted to third parties except OpenAI for embeddings and generation
- Users shall have full control over their data

NFR5.3 - Input Validation

- All user inputs shall be validated before processing
- SQL injection shall be prevented through parameterized queries
- File paths shall be validated to prevent directory traversal attacks

NFR6: Scalability

NFR6.1 - Knowledge Base Size

- The system shall support knowledge bases with up to 1,000 documents
- Document size shall be limited to 50MB to ensure reasonable processing time

NFR6.2 - Session History

- The system shall maintain session history for the lifetime of a session
- Database queries shall remain performant with large message tables

NFR7: Compatibility

NFR7.1 - Platform Support

- The system shall run on macOS, Windows, and Linux
- JavaFX 26 or higher shall be required
- Java 17 or higher shall be required

NFR7.2 - Document Format Support

- The system shall correctly extract text from all specified document formats
- Extraction shall handle various encodings (UTF-8, ASCII, etc.)
- Invalid documents shall be rejected with appropriate messages

NFR7.3 - API Compatibility

- The system shall use Langchain4j 1.8.0 for OpenAI integration
- The system shall be compatible with OpenAI API

Use Case Diagram and Descriptions

The following use case diagram illustrates the primary interactions between users and the RAG system. The diagram encompasses all major functional requirements, demonstrating the relationships between actors, use cases, and system boundaries.

